

BÁO CÁO CÁCH SỬ DỤNG VÀ MONITOR REDIS TRONG PRODUCTION BACKEND LAB

Phạm vi: Full lab khởi động bằng make up-full

Thành phần được phân tích: API server, consumer worker, Redis primary, replica và Sentinel

Mục tiêu: Giải thích vai trò thực tế của Redis, các đoạn code liên quan, luồng dữ liệu và phương pháp monitor

1. Tóm Tắt Điều Hành

Redis trong lab này không chỉ đóng vai trò cache. Nó đảm nhận bốn chức năng:

Chức năng	Key	Client sử dụng	Đặc điểm
Rate-limit store	rate:<client_id>:<minute>	API server	Counter tạm thời, TTL 60 giây
Idempotency store	idempotency:<key>	API server	Cache HTTP response, TTL 3600 giây
Operational state store	job:<order_id>	API server và worker	Trạng thái hiện tại của order, TTL 86400 giây
Read cache	job:<order_id>	API server	Trả trạng thái nhanh trước khi fallback sang Elasticsearch

Redis là nguồn dữ liệu ưu tiên khi API cần trả lời trạng thái hiện tại của order. Tuy nhiên, Redis không phải cơ sở dữ liệu nghiệp vụ lâu dài vì mọi key ứng dụng đều có TTL và có thể bị xóa theo eviction policy.

Phân chia trách nhiệm trong lab:

RabbitMQ = giữ và vận chuyển công việc cần xử lý
Redis = trạng thái vận hành hiện tại và dữ liệu tạm thời
Elasticsearch = lưu kết quả processed lâu dài và hỗ trợ tìm kiếm

Thiết kế này phù hợp để minh họa backend bất đồng bộ, nhưng chưa đủ chặt chẽ cho production. Một hệ thống production thường dùng PostgreSQL hoặc cơ sở dữ liệu giao dịch tương đương làm source of truth.

2. Kiến Trúc Và Vai Trò Các Thành Phần

2.1 Luồng tổng quát

```
HTTP Client
|
v
API Server
|-- Redis: rate limit, idempotency, trạng thái queued
|-- RabbitMQ: publish order
|
v
Consumer Worker
|-- Redis: processing, processed hoặc dead_lettered
|-- Elasticsearch: lưu order processed
```

2.2 Phân biệt HTTP client và Redis client

client/load_test.py là HTTP client dùng để chạy scenario test:

```
requests.post(
    f"{API}/orders",
    json=payload,
    headers={"Idempotency-Key": key} if key else {},
    timeout=10,
)
```

HTTP client không kết nối trực tiếp tới Redis.

API server và consumer worker mới là Redis clients. Hai service này import thư viện redis-py, khởi tạo object redis.Redis và gửi command tới Redis primary.

2.3 Redis trong full lab

make up-full khởi động:

- redis: primary được ứng dụng kết nối trực tiếp.
- redis-replica: sao chép dữ liệu từ primary.
- redis-sentinel: theo dõi primary với tên labmaster.

Cấu hình ứng dụng:

```
REDIS_URL: redis://redis:6379/0
```

API và worker kết nối trực tiếp tới hostname redis. Chúng không sử dụng Sentinel discovery để tìm primary mới khi failover.

3. Khởi Tạo Redis Client

3.1 Dependency

API và worker sử dụng:

```
redis==6.1.0
```

Các file dependency:

- api-server/requirements.txt
- consumer-worker/requirements.txt

3.2 Đọc cấu hình kết nối

Trong shared/common.py:

```
def env(name, default):
    return os.getenv(name, default)
```

```
REDIS_URL = env("REDIS_URL", "redis://redis:6379/0")
```

URL có ý nghĩa:

```
redis://redis:6379/0
|   |   |   |
|   |   |   +-- database 0
|   |   +----- port Redis trong Docker network
|   +----- hostname Redis primary
+----- giao thức Redis không dùng TLS
```

3.3 Factory tạo Redis client

API và worker dùng chung hàm:

```
def redis_client():
    return redis.Redis.from_url(
        REDIS_URL,
        decode_responses=True,
        socket_timeout=2,
    )
```

Ý nghĩa:

- `Redis.from_url`: parse URL và tạo client cùng connection pool.
- `decode_responses=True`: chuyển response từ bytes thành Python str.
- `socket_timeout=2`: command chờ response tối đa hai giây.

Kết nối TCP thường được mở khi command đầu tiên chạy, không phải ngay lúc gọi `Redis.from_url`.

3.4 Đánh giá connection pool hiện tại

Code hiện tại tạo client mới nhiều lần:

```
redis_client().get(...)
redis_client().setex(...)
```

Mỗi object Redis có connection pool riêng. Cách này chạy được trong lab nhưng làm giảm khả năng tái sử dụng connection và khó kiểm soát connection count khi tải tăng.

Thiết kế production phù hợp hơn:

```
REDIS = redis.Redis.from_url(
    REDIS_URL,
    decode_responses=True,
    socket_connect_timeout=2,
    socket_timeout=2,
    health_check_interval=30,
)
```

```
def redis_client():
    return REDIS
```

Một client/pool dùng chung cho mỗi process giúp quản lý connection, timeout và health check nhất quán.

4. API Server Sử Dụng Redis

Endpoint tạo order khởi tạo Redis client ở đầu request:

```
@app.post("/orders", status_code=202)
```

```
def create_order(...):
    r = redis_client()
```

Client `r` được tái sử dụng cho rate limit, idempotency và ghi trạng thái trong cùng request.

4.1 Rate limit

API tạo key theo `client_id` và cửa sổ phút:

```
rate_key = f"rate:{order.client_id}:{int(time.time() // 60)}"
count = r.incr(rate_key)
if count == 1:
    r.expire(rate_key, 60)
```

Luồng command:

```
Request đầu tiên:
INCR rate:<client>:<minute> -> value = 1
EXPIRE rate:<client>:<minute> 60
```

```
Request tiếp theo:
INCR rate:<client>:<minute> -> value tăng dần
```

API từ chối request khi vượt giới hạn:

```
if count > int(os.getenv("RATE_LIMIT_PER_MINUTE", "10")):
    raise HTTPException(429, "rate limit exceeded")
```

Rủi ro: INCR và EXPIRE là hai command riêng. Nếu process lỗi giữa hai command, counter có thể tồn tại mà không có TTL. Production nên dùng Lua script hoặc cơ chế atomic tương đương.

4.2 Idempotency

API đọc cache khi request có Idempotency-Key:

```
cached = r.get(f"idempotency:{idempotency_key}")
if cached:
    result = json.loads(cached)
    return result
```

Nếu GET hit, API trả lại response cũ và không publish thêm order vào RabbitMQ.

Sau khi publish order thành công, API cache response:

```
r.setex(
    f"idempotency:{idempotency_key}",
    3600,
    json.dumps(result),
)
```

SETEX ghi value và TTL trong cùng một command. Key idempotency tự hết hạn sau một giờ.

Rủi ro: kiểm tra GET, publish RabbitMQ và ghi SETEX không phải một transaction. Hai request đồng thời có cùng idempotency key vẫn có thể cùng thấy cache miss và cùng publish order.

4.3 Trạng thái queued

Sau khi RabbitMQ xác nhận publish, API ghi trạng thái:

```
result = {
```

```

    "order_id": order_id,
    "job_id": order_id,
    "request_id": request_id,
    "trace_id": trace_id,
    "status": "queued",
}

r.setex(
    f"job:{order_id}",
    86400,
    json.dumps(result),
)

```

Key job:<order_id> tồn tại tối đa 24 giờ và mô tả trạng thái hiện tại của order.

4.4 Đọc trạng thái order

```

@app.get("/orders/{order_id}")
def get_order(order_id: str):
    value = redis_client().get(f"job:{order_id}")
    if value:
        return json.loads(value)
    return es_client().get(index="orders", id=order_id)["_source"]

```

Luồng đọc:

```

GET /orders/<id>
|
+-- Redis hit --> trả trạng thái hiện tại
|
+-- Redis miss --> fallback sang Elasticsearch

```

Điều này làm job:* đồng thời là operational state store và read cache.

4.5 Readiness check

Endpoint /ready kiểm tra Redis bằng:

```
"redis": lambda: redis_client().ping()
```

Nếu PING lỗi hoặc timeout, API đánh dấu Redis không sẵn sàng và trả HTTP 503.

5. Consumer Worker Sử Dụng Redis

Worker nhận message từ RabbitMQ và cập nhật trạng thái cùng key job:<order_id>.

5.1 Bắt đầu xử lý

```

redis_client().setex(
    f"job:{data['id']}",
    86400,
    json.dumps(**data, "status": "processing"),
)

```

5.2 Xử lý thành công

```

result = {
    **data,
    "status": "processed",
}

```

```

    "processed_at": datetime.now(timezone.utc).isoformat(),
}

es_client().index(
    index="orders",
    id=data["id"],
    document=result,
    refresh=False,
)

redis_client().setex(
    f"job:{data['id']}",
    86400,
    json.dumps(result),
)

```

Elasticsearch lưu kết quả lâu dài. Redis giữ trạng thái hiện tại để API trả lời nhanh.

5.3 Xử lý thất bại

Khi chưa hết số lần retry, worker republish message vào retry queue. Mỗi lần message quay lại, worker ghi trạng thái processing.

Khi hết retry:

```

redis_client().setex(
    f"job:{data['id']}",
    86400,
    json.dumps({
        **data,
        "status": "dead_lettered",
        "error": str(exc),
    }),
)

```

Trạng thái dead_lettered chỉ tồn tại trong Redis và message trong DLQ; nó không được index vào Elasticsearch. Khi TTL Redis hết, endpoint order không còn thấy trạng thái dead-letter.

6. Luồng Hoạt Động End-To-End

6.1 Order thành công

1. HTTP client gửi POST /orders
2. API:
 - INCR + EXPIRE rate key
 - GET idempotency key nếu có
3. API publish message vào RabbitMQ
4. API:
 - SETEX job:<id> = queued
 - SETEX idempotency:<key> nếu có
5. API trả HTTP 202
6. Worker nhận message
7. Worker SETEX job:<id> = processing
8. Worker index order processed vào Elasticsearch
9. Worker SETEX job:<id> = processed
10. Worker ACK message

6.2 Request idempotency hit

1. Client gửi lại request với cùng Idempotency-Key

2. API INCR rate key
3. API GET idempotency:<key>
4. Redis trả response JSON cũ
5. API trả cùng order_id
6. Không publish thêm message vào RabbitMQ

6.3 Poison message

1. API tạo job queued và publish message
2. Worker ghi processing
3. Worker phát sinh simulated error
4. Message đi qua retry queue rồi quay lại worker
5. Sau khi hết retry, message vào DLQ
6. Worker ghi job:<id> = dead_lettered

6.4 Lifecycle của key

Key	Được tạo khi nào	Được cập nhật khi nào	Kết thúc
rate:*	Request đầu tiên trong phút	Mọi request cùng client/phút	Tự hết hạn sau 60 giây
idempotency:*	Order có idempotency key publish thành công	Không cập nhật	Tự hết hạn sau 3600 giây
job:*	API publish order thành công	Worker processing/processed/dead-lettered	Tự hết hạn sau 86400 giây

7. Redis Là Cache Hay Cơ Sở Dữ Liệu?

Redis trong lab là một kho dữ liệu đa vai trò:

- idempotency:* là cache response và idempotency store.
- rate:* là counter store tạm thời.
- job:* là operational state store và cache đọc nhanh.

Redis trông giống cơ sở dữ liệu chính vì API đọc job:* trước và worker liên tục cập nhật nó. Tuy nhiên, nó không phải source of truth hoàn chỉnh:

- Mọi key có TTL.
- Cấu hình dùng allkeys-lru, nên key có thể bị evict khi thiếu memory.
- dead_lettered không được lưu vào Elasticsearch.
- Redis mất dữ liệu có thể khiến API tạm thời trả 404.
- Elasticsearch chỉ nhận order đã xử lý thành công.

Kiến trúc production phổ biến hơn:

PostgreSQL = source of truth cho order và trạng thái
 Redis = cache, rate limit, idempotency tạm thời
 RabbitMQ = vận chuyển công việc
 Elasticsearch = search index

8. Chạy Full Lab Và Tạo Redis Traffic

Khởi động:

```
make down
make up-full
FULL="docker compose -f docker-compose.full.yml"
```

Kiểm tra readiness:

```
curl -sS http://localhost:8000/ready | jq
```

Build HTTP test client:

```
$FULL --profile tools build client
```

Tạo các luồng Redis:

```
$FULL --profile tools run --rm client normal
$FULL --profile tools run --rm client idempotency
$FULL --profile tools run --rm client rate-limit --count 12
$FULL --profile tools run --rm client dlq
```

HTTP test client chỉ gọi API. Các Redis command được tạo bởi API server và consumer worker.

9. Monitor Redis Trong Lab

9.1 Quan sát command do code tạo

Terminal 1:

```
FULL="docker compose -f docker-compose.full.yml"
$FULL exec redis redis-cli MONITOR
```

Terminal 2:

```
$FULL --profile tools run --rm client idempotency
$FULL --profile tools run --rm client rate-limit --count 12
$FULL --profile tools run --rm client dlq
```

Đối chiếu command:

Command	Nguồn
PING	API /ready
INCR rate:...	API rate limit
EXPIRE rate:... 60	API đặt TTL cho counter mới
GET idempotency:...	API kiểm tra request trùng
SETEX idempotency:... 3600	API cache response
SETEX job:... 86400	API hoặc worker cập nhật trạng thái
GET job:...	API đọc trạng thái order

MONITOR chỉ phù hợp cho lab hoặc điều tra ngắn hạn vì có overhead và hiển thị dữ liệu trong command.

9.2 Monitor connection

```
$FULL exec redis redis-cli INFO clients
$FULL exec redis redis-cli CLIENT LIST
```


Các field quan trọng:

Field	Ý nghĩa
connected_clients	Tổng số client đang kết nối
blocked_clients	Client chờ blocking command
addr	Địa chỉ client
idle	Thời gian connection không hoạt động
cmd	Command gần nhất

Nếu connected_clients tăng liên tục khi tải tăng và không giảm, cần kiểm tra việc tạo client/pool mới trong code.

9.3 Dashboard terminal của lab

Terminal 1:

```
make perf-full-observe
```

Terminal 2:

```
make perf-full-http
```

Script observer hiển thị CPU, RAM, network I/O, block I/O và các Redis metric quan trọng.

9.4 Stats, memory và commandstats

```
$FULL exec redis redis-cli INFO stats
$FULL exec redis redis-cli INFO memory
$FULL exec redis redis-cli INFO keyspace
$FULL exec redis redis-cli INFO commandstats
```

Metric	Ý nghĩa	Dấu hiệu cần điều tra
instantaneous_ops_per_sec	Command mỗi giây	Không tăng nhưng HTTP latency tăng
keyspace_hits	Số lần đọc key thành công	Đánh giá hiệu quả cache
keyspace_misses	Số lần không tìm thấy key	Tăng mạnh do key hoặc TTL không phù hợp
evicted_keys	Key bị xóa vì memory pressure	Giá trị tăng lớn hơn 0
expired_keys	Key tự xóa vì hết TTL	Xác nhận TTL hoạt động
total_error_replies	Response lỗi từ Redis	Tăng nhanh
used_memory_human	Memory Redis đang dùng	Tiến gần giới hạn host/container
mem_fragmentation_ratio	RSS so với allocated memory	Cao kéo dài cần điều tra

INFO commandstats cho biết số lần gọi và thời gian xử lý từng command như get, setex, incr, expire và ping.

9.5 Slowlog và latency

Cấu hình redis/redis.conf:

```
latency-monitor-threshold 10
slowlog-log-slower-than 1000
slowlog-max-len 256
```

Kiểm tra:

```
$FULL exec redis redis-cli SLOWLOG LEN
$FULL exec redis redis-cli SLOWLOG GET 20
$FULL exec redis redis-cli LATENCY LATEST
$FULL exec redis redis-cli LATENCY DOCTOR
$FULL exec redis redis-cli --latency
```

Nếu GET, SETEX hoặc INCR xuất hiện trong slowlog với latency cao, cần kiểm tra CPU, memory pressure, persistence I/O và tải tổng thể.

9.6 Monitor TTL và key lifecycle

Chạy scenario:

```
$FULL --profile tools run --rm client idempotency
```

Quan sát key do code tạo:

```
$FULL exec redis redis-cli --scan --pattern 'rate:*'
$FULL exec redis redis-cli --scan --pattern 'idempotency:*'
$FULL exec redis redis-cli --scan --pattern 'job:*'
```

Kiểm tra TTL:

```
$FULL exec redis redis-cli TTL "rate:REPLACE_WITH_KEY"
$FULL exec redis redis-cli TTL "idempotency:REPLACE_WITH_KEY"
$FULL exec redis redis-cli TTL "job:REPLACE_WITH_ORDER_ID"
```

Không dùng KEYS * trên production vì command này có thể block Redis. Dùng SCAN.

9.7 Monitor replication và Sentinel

```
$FULL exec redis redis-cli INFO replication
$FULL exec redis-replica redis-cli INFO replication
$FULL exec redis-sentinel redis-cli -p 26379 SENTINEL master labmaster
$FULL exec redis-sentinel redis-cli -p 26379 SENTINEL replicas labmaster
```

Kỳ vọng:

```
primary: role:master, connected_slaves:1
replica: role:slave, master_link_status:up
```

Sentinel có thể promote replica, nhưng API và worker không tự chuyển sang primary mới vì chúng kết nối trực tiếp tới hostname redis.

10. Cấu Hình Persistence Và Memory

Cấu hình Redis primary:

```
appendonly yes
appendfsync everysec
save 60 1000
maxmemory-policy allkeys-lru
```

Ý nghĩa:

- AOF ghi lại command thay đổi dữ liệu.

- appendfsync everysec có thể mất khoảng một giây dữ liệu khi host lỗi đột ngột.
- RDB snapshot được tạo khi có ít nhất 1000 thay đổi trong 60 giây.
- allkeys-lru cho phép Redis evict bất kỳ key nào khi chạm maxmemory.

Kiểm tra persistence:

```
$FULL exec redis redis-cli INFO persistence
```

Cần theo dõi:

- aof_enabled
- aof_last_write_status
- aof_last_bgrewrite_status
- rdb_last_bgsave_status

File cấu hình lab chưa đặt maxmemory cụ thể. Vì vậy cần theo dõi đồng thời memory Redis và memory container/host.

11. Rủi Ro Và Khuyến Nghị Production

Vấn đề hiện tại	Tác động	Khuyến nghị
Tạo Redis client/pool nhiều lần	Connection khó kiểm soát	Dùng một client/pool chung cho mỗi process
INCR và EXPIRE tách rời	Counter có thể thiếu TTL	Dùng Lua atomic
Idempotency không atomic	Request đồng thời có thể tạo order trùng	Dùng lock/claim atomic hoặc lưu source of truth
Redis giữ trạng thái quan trọng có TTL	Trạng thái biến mất sau TTL	Lưu trạng thái vào database giao dịch
dead_lettered không vào Elasticsearch	API mất trạng thái sau TTL	Persist trạng thái lỗi lâu dài
Ứng dụng không Sentinel-aware	Không tự failover	Dùng Sentinel-aware hoặc Cluster client
Chỉ có một Sentinel	Không đủ quorum production	Chạy tối thiểu ba Sentinel độc lập
Không authentication/TLS	Không an toàn ngoài lab	Dùng ACL, auth, TLS và private network
Không đặt maxmemory	Rủi ro OOM host/container	Đặt giới hạn và eviction policy rõ ràng

12. Kết Luận

Redis trong Production Backend Lab là một operational data store tốc độ cao, không chỉ là cache. API dùng Redis để rate limit, idempotency và đọc trạng thái nhanh. Worker dùng Redis để cập nhật lifecycle của order. Redis primary có replica, Sentinel, AOF, RDB, slowlog và latency monitor để phục vụ thực hành vận hành.

Thiết kế hiện tại minh họa rõ cách Redis phối hợp với RabbitMQ và Elasticsearch. Điểm quan trọng nhất là Redis đang giữ trạng thái có giá trị nghiệp vụ nhưng không phải source of truth bền vững. Khi đưa mô hình này lên production, cần bổ sung database giao dịch, client/pool dùng chung, atomic operations, failover-aware client, security và monitoring tập trung.